

NetraSetu — SIH demo video script

Target runtime: **6 minutes** (fits the 5–7 minute window with margin). Two people, two separate recordings, combined in editing — read the recording setup below before either of you starts.

Recording setup & roles

- **Tanuj's machine runs the whole prototype** (central web, PHC desktop web, MATLAB/Simulink) and stays the single source of truth for what's on screen — nobody records against a second, independently-set-up copy, so there's no risk of the video showing a different state than the live system.
 - **Tanuj sets up AnyDesk** on his machine and shares the session with **Parth**.
 - **Parth connects over AnyDesk, does the voiceover, and drives the UI** for everything except the mobile app: PHC desktop, central web (ophthalmologist + district admin), and the live Simulink model. He is narrating a system he didn't build, so every section below includes a short **"Why this matters"** note he can absorb beforehand — say the *Say*: lines close to verbatim, since they're phrased to be accurate without overclaiming.
 - **Tanuj records the mobile segment himself**, separately, on his own phone, at his own pace — not over AnyDesk. That clip gets spliced into the timeline at the marked point.
 - **Before Parth's recording session**: Tanuj runs `node scripts/demo-reset.js` (~4 min, not part of the video) and keeps its output terminal visible to himself (Parth doesn't need it, just the credentials relayed once) — it prints every login needed below. Load each web app once, wait ~10s, and reload before recording starts (Vite's first-request dependency optimization eats anything typed during it). Have MATLAB already open with `simulink-model/netraSetuPipeline.slx` loaded (but not yet run) in a window ready to switch to, so Parth isn't waiting on MATLAB's own startup time on camera.
 - **Say the number on screen, not a rounded one** — every metric a judge might screenshot should match the live app exactly.
-

[PARTH] 0:00 – 0:20 — Cold open

Voiceover over a static title card or the login screen.

"India has roughly one ophthalmologist per 100,000 rural people, but diabetic retinopathy affects nearly a fifth of the country's diabetic adults. NetraSetu lets a minimally-trained technician at a rural clinic screen a patient in minutes, with two independent AI models cross-checking each other, and a specialist confirming every positive result before it reaches the patient."

[PARTH] 0:20 – 1:00 — PHC login and patient registration (desktop, :5173)

Why this matters: this is the only screen a rural clinic technician — not a doctor, not IT staff — ever touches. Everything here has to be simple enough for that person and still capture what the AI pipeline needs.

1. Log in as `technician` with the password `demo-reset` printed.
2. **Say**: "Every technician login is real — bcrypt-checked, session-based, not a demo shortcut."
3. New patient: fill name, age, gender, contact number, address. Answer the risk questionnaire (known diabetic, years since diagnosis, glycemic control, BP, symptoms).
4. **Say**: "This questionnaire can't be skipped, and it only ever nudges the AI's confidence — it never overrides what the image shows."
5. Tick informed verbal consent, click **INITIATE CAPTURE**.

[PARTH] 1:00 – 2:10 — Capture, quality-gate reject, retake, accept

Why this matters: a bad photo sent to a specialist wastes a review slot on nothing. Catching it before the patient leaves the chair is the whole point of doing this check locally.

1. Choose eye (Left/OS) and camera preset. Import `idrid_164_bad_blur_dark.jpg` (`tests/fixtures/`). Click **RUN QUALITY CHECK**.
2. While it runs (~10–15s, a real `matlab -batch` call): **say:** "This quality check runs on-device, on real MATLAB code — not a placeholder. A rural clinic can't afford to send a bad photo to the cloud and find out five minutes later."
3. **Expect:** Quality fail, "Image below diagnostic threshold," engine **MATLAB**, issue "Image is too dark." Click **RETAKING IMAGE**.
4. Import `idrid_010_good_pass_w1800.jpg` , **RUN QUALITY CHECK** again.
5. **Expect:** Quality pass. Click **ACCEPT & CONTINUE**.
6. Complete the capture-metadata questionnaire (dilated, indoor clinic, none noticed), click **CLEAR**, then **SAVE & SYNC TO SERVER**.
7. **Say:** "The image, both questionnaires, and the quality result sync to central now — or queue locally the moment connectivity drops."

[PARTH] 2:10 – 2:35 — Offline resilience (compressed beat)

Why this matters: rural connectivity isn't a hypothetical edge case here — it's the normal operating condition this system was designed around, not an afterthought bolted on.

1. Terminal: `node scripts/demo-offline.js stop` . Wait ~15s — header turns amber, **OFFLINE**.
2. **Say, over the amber header:** "If the network drops mid-shift, nothing is lost and nothing pretends to have synced — it queues, ordered by urgency, and sends the moment connectivity returns."
3. `node scripts/demo-offline.js restore` . Wait for **ONLINE** (~10s). Don't wait for the full RESULT READY transition on camera — cut once it reads ONLINE.

[PARTH] 2:35 – 4:10 — Central: dual-branch grading, Grad-CAM, confidence tiers (: 5174)

Why this matters: this is the core clinical-safety idea of the whole project — one model alone can be wrong with no way to tell; two independent models that either agree (real evidence) or disagree (mandatory human review) is a genuinely different safety property, not a marketing line.

1. Log out of the intro, choose **OPHTHALMOLOGIST**, log in with `ophthalmologist@demo.netrasetu.local` and its printed password.
2. **CASES** queue. **Say:** "Every referable and uncertain case lands here — the queue is sorted so the model's least-confident cases surface first, not the most severe. High-confidence, low-risk cases never reach a human at all."
3. Open a referable row. **Say, pointing at the two grade columns:** "Two independent models grade every image — a deep-learning classifier, and a separate rule engine implementing the actual clinical ICDR criteria by counting lesions per quadrant. When they agree, that's real evidence the grade is right."
4. Open the flagged disagreement case (the reset seeds one). **Say:** "When they disagree, the case is forced to mandatory review — a reviewer can't just click Confirm, they have to pick an explicit final grade. Disagreement is treated as a safety signal, not noise to average away."
5. Click **GRAD-CAM OFF** to turn the heatmap on. **Say:** "This shows exactly which pixels drove the AI's decision — not a black box."
6. Scroll to lesion evidence (red/bright lesions per quadrant) and the confidence tier. **Say:** "Every case also gets a statistically calibrated confidence tier, not just a raw softmax score — that's what decides whether a case needs full manual review or can auto-clear safely."

[PARTH] 4:10 – 4:50 — Confirm and override

1. On the agreeing case: **CLINICAL DECISION & SAFETY AUDIT** → **CONFIRM** → **SUBMIT CONFIRM**. Green flash, case leaves the queue.
2. Open the disagreement case: **OVERRIDE**, pick the corrected grade and a reason, submit. **Say:** "Every override is captured with its reason — that's the raw material for the system's continual-learning loop down the line."

[PARTH] 4:50 – 5:15 — Referral tracking (district admin)

1. Log out, choose **DISTRICT WORKER**, log in as `admin@demo.netraSetu.local`.
2. Click **REFERRALS**. **Say what's actually on screen** — one of two honest outcomes, both correct system behavior, so narrate whichever one you see rather than assuming: *if* the row shows an SMS actually sent, say "every confirmed or overridden referable case becomes a tracked referral, and the patient is notified automatically." *If* it shows "manual follow-up" (SMS not configured), say "the system detected it couldn't send the SMS and flagged it for a human to call instead, rather than silently doing nothing." Either way: **"Nothing about patient follow-up is left to chance."**

[PARTH] 5:15 – 5:35 — Live Simulink resource model

Why this matters: this isn't a static slide — it's a real discrete-event simulation the team built, showing the system thinks about capacity planning at district scale, not just per-patient grading.

1. Switch to the pre-loaded MATLAB window (`simulink-model/netraSetuPipeline.slx`). Click **Run**.
2. **Say, while it runs:** "This is a real discrete-event simulation — patient images flowing through capture, network transmission, and ophthalmologist review as a queueing system, at district scale. It's what generates the resource recommendations district admins see: where the bottleneck actually is, not a guess."
3. Briefly point at the moving entities / queue visualization if the model shows one live.

[TANUJ — separate recording, own phone] 5:35 – 5:55 — Mobile app (second front-end)

Recorded independently by Tanuj, not over AnyDesk. Spliced in here.

1. Screen-record: registration → gallery/lens import of a fundus photo → on-device quality check → sync.
2. **Voiceover:** "The same workflow also runs as a phone app for a technician without a dedicated station — same backend, same rules, same safety guarantees. Not a separate, simplified path."

[PARTH] 5:55 – 6:00 — Close

"Two independent AI models, a specialist always in the loop, offline-first for real rural connectivity, and every decision explainable down to the pixel. That's NetraSetu."

If you have the extra 60–90s (7-minute cut)

Insert after 4:10 (before override), ~45s: PHC Health screen — show a PHC gone silent past threshold, explaining the system-health alerting most screening tools don't have. Or insert after 2:35, ~30s: show the local queue's storage-pressure warning / manual export utility, since "multi-day outage" is a real rural-India scenario, not a hypothetical.

Recovery notes (only matters if something breaks on camera)

- Quality check stuck / MATLAB start-up failure: click the retry button that appears; the saved image re-checks without a new photo.
- "502 Bad Gateway" on central login: the web app is pointed at the wrong backend port — means `demo-reset` wasn't re-run after a `.env` change; Tanuj re-runs it before the next take.
- A case still shows `processing` after 30s in the review queue: wait, then reload — grading is a real ~21–25s MATLAB + segmentation pipeline, not instant by design.
- AnyDesk lag on a click: pause narration briefly rather than clicking twice — a doubled click can trigger an unintended action (e.g. a second form submit).